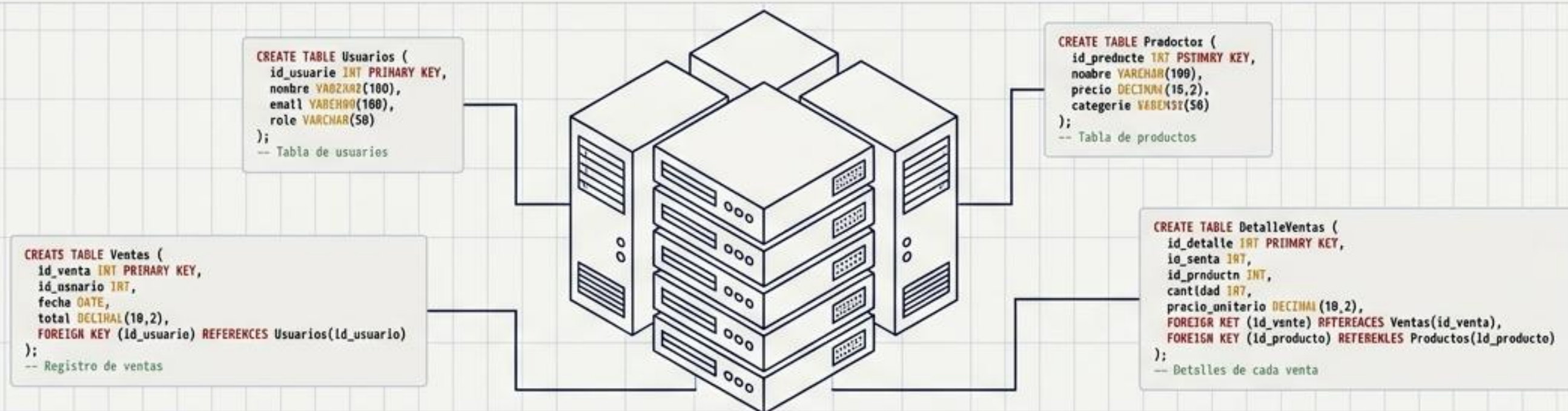
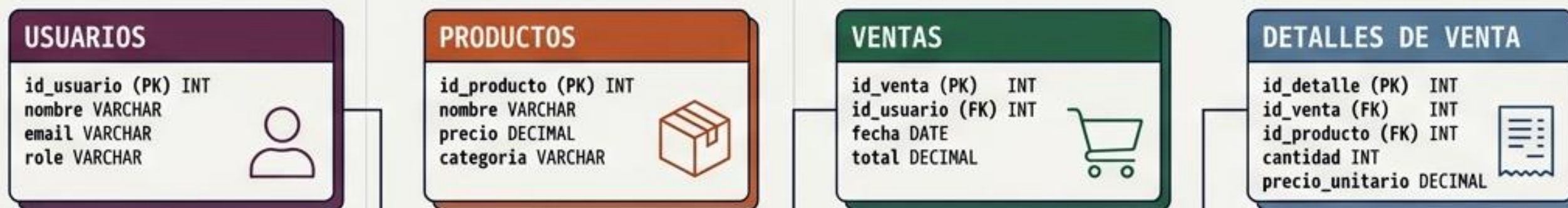




Arquitectura de una Base de Datos Relacional

> Proyecto: almacen_ropa // De código SQL a estructura visual



Tablas Catálogo (Independientes)

Datos maestros que existen por sí solos. Representan entidades físicas o personas.



`usuarios`



`productos`

Tablas Transaccionales (Eventos)

Registran eventos que ocurren en el tiempo. Dependen de que existan catálogos previos.



`ventas`



`detalle\_ventas`

Tipo de Dato	Uso en nuestro sistema	El Porqué
INT	IDs numéricos y cantidad de stock.	// Usamos números enteros porque no se venden fracciones de ropa.
VARCHAR	Nombres, usuarios, rutas de imagen.	// Límite de texto variable. Para imágenes guardamos solo la ruta (ej: 'foto1.jpg'), no la imagen real.
DECIMAL(10, 2)	Precios y totales monetarios.	// 10 dígitos en total con 2 decimales exactos para los centavos. Evita errores de redondeo.
DATETIME	Fecha de ventas.	// Captura automáticamente el instante exacto en el servidor.
ENUM	Roles del sistema.	// Seguridad estricta: restringe los valores permitidos únicamente a 'admin' o 'empleado'.

```
CREATE TABLE usuarios (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  nombre VARCHAR(100),  
  usuario VARCHAR(50) UNIQUE,  
  password VARCHAR(255),  
  rol ENUM('admin', 'empleado')  
);
```

Identificador único
que aumenta solo
y no se repite.

Impide que dos
personas tengan
el mismo nombre
de usuario.

255 caracteres
asegura que
quepa el hash de
seguridad.




```
CREATE TABLE productos (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  nombre VARCHAR(100),  
  precio DECIMAL(10, 2),  
  stock INT,  
  imagen VARCHAR(255)  
);
```

Llave primaria:
identificación
única para cada
prenda.

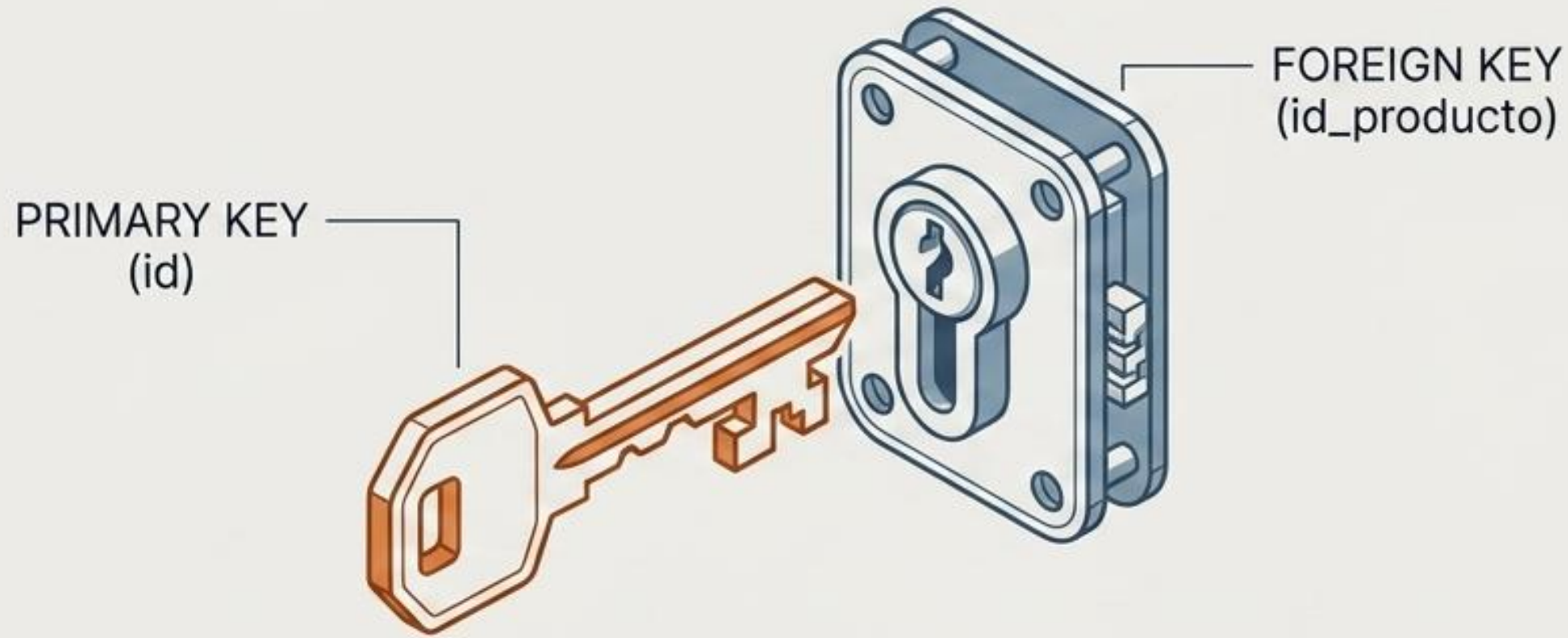
Cantidad física
en bodega
(número entero).

No subas el
archivo a la BD.
Guarda solo el
nombre (ej:
'camisa.jpg').



productos				
id	nombre	precio	stock	imagen

Reglas de Integridad Referencial



Una clave foránea (FOREIGN KEY) es un candado de seguridad.
La regla es absoluta: No puedes registrar una venta de un producto cuyo ID no existe previamente en la tabla de productos.
Esto garantiza que tu base de datos nunca tenga datos huérfanos.


```
CREATE TABLE ventas (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  fecha DATETIME DEFAULT CURRENT_TIMESTAMP,  
  total DECIMAL(10, 2),  
  id_vendedor INT,  
  FOREIGN KEY (id_vendedor) REFERENCES usuarios(id)  
);
```

```
// Capturas de ventas  
id INT AUTO_INCREMENT,  
fecha DATETIME DEFAULT,  
total DECIMAL(10, 2),  
id_vendedor INT,  
FOREIGN KEY (id_vendedor) REFERENCES usuarios(id)  
);
```

Captura automáticamente
la fecha y hora exacta del
servidor al momento de
vender.

Clave foránea que
conecta esta cabecera
de factura con el ID de
un empleado real en la
tabla `usuarios`.



FOREIGN KEY
(id_vendedor)

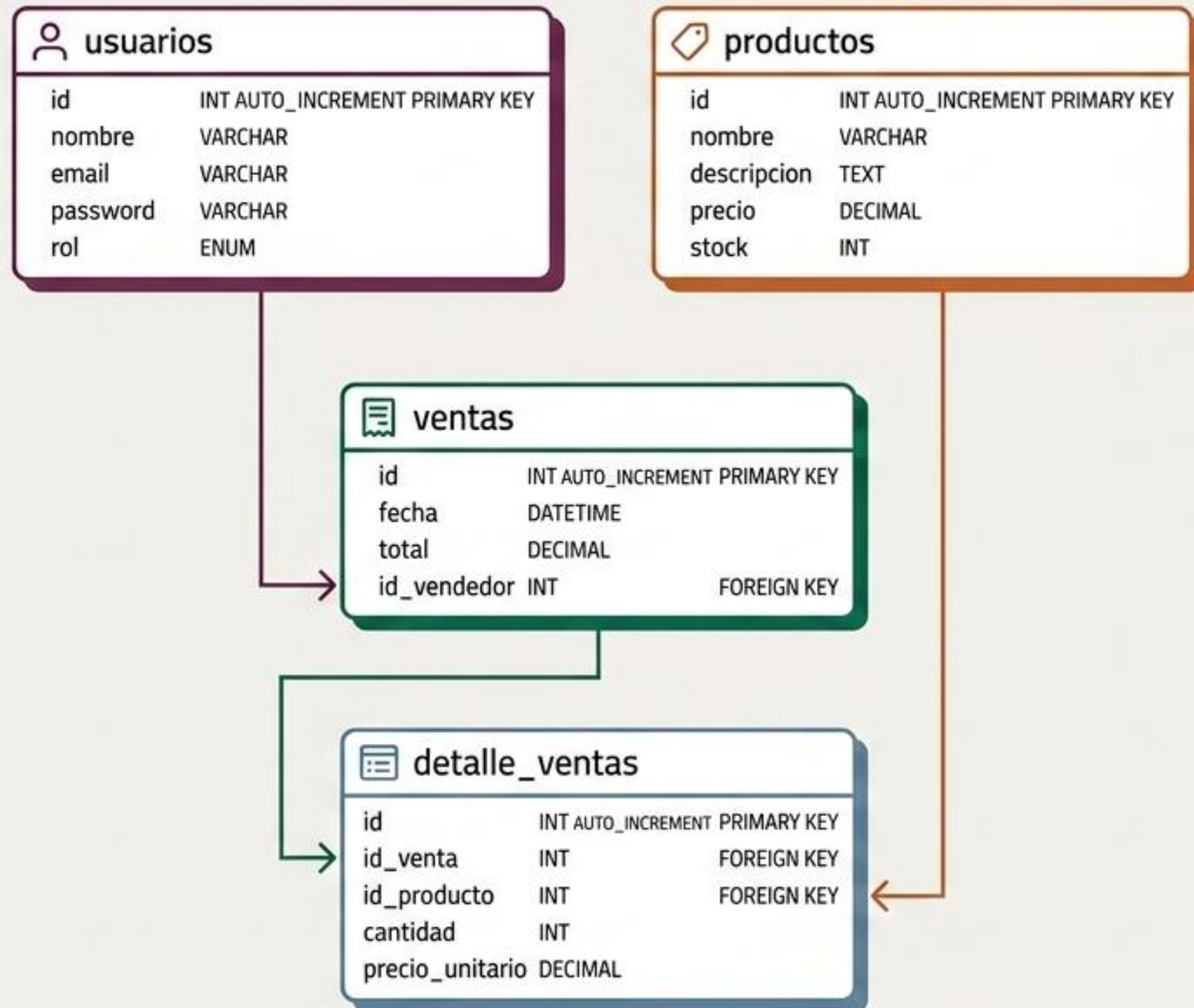
```
CREATE TABLE detalle_ventas (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  id_venta INT,  
  id_producto INT,  
  cantidad INT,  
  precio_unitario DECIMAL(10, 2),  
  FOREIGN KEY (id_venta) REFERENCES ventas(id),  
  FOREIGN KEY (id_producto) REFERENCES productos(id)  
);
```

Regla de integridad: No puede existir un renglón de detalle si no hay una factura madre.

Asegura que no se vendan prendas inexistentes en el inventario.

Se guarda el precio *al momento de la venta*. Es vital para mantener el registro histórico aunque el precio del catálogo cambie en el futuro.

detalle_ventas					
id					
id_venta					
id_producto					
cantidad					
precio_unitario					



El modelo está completo. Las tablas independientes alimentan a las transaccionales, creando un ecosistema de datos protegido por reglas de integridad.